

## QUESTION 2

```
import random
import math

# -----
# Landscape definition (1-indexed positions)
# -----
LANDSCAPE = [5, 8, 6, 12, 9, 7, 17, 14, 10, 6, 19, 15, 11, 8]
# Index:    0 1 2 3 4 5 6 7 8 9 10 11 12 13
# State:    1 2 3 4 5 6 7 8 9 10 11 12 13 14

GLOBAL_MAX_STATE = 11 # 1-indexed (f=19)
GLOBAL_MAX_IDX = 10 # 0-indexed

# -----
# Q1 helper: First-Choice Hill Climbing
# -----
def first_choice_hc(landscape, start):
    """
    Moves to the FIRST neighbour that strictly improves the current value.
    Returns (terminal_index, path_of_indices).
    """
    current = start
    path = [current]
    n = len(landscape)
    while True:
        improved = False
        neighbours = []
        if current > 0:
            neighbours.append(current - 1)
        if current < n - 1:
            neighbours.append(current + 1)
        for nb in neighbours:
            if landscape[nb] > landscape[current]:
                current = nb
                path.append(current)
                improved = True
                break
        if not improved:
            break
    return current, path
```

```

# -----
# Q1 helper: Stochastic Hill Climbing
# -----
def stochastic_hc(landscape, start):
    """
    Collects all improving neighbours; chooses one uniformly at random.
    Returns (terminal_index, path_of_indices).
    """
    current = start
    path = [current]
    n = len(landscape)
    while True:
        improving = []
        if current > 0 and landscape[current - 1] > landscape[current]:
            improving.append(current - 1)
        if current < n - 1 and landscape[current + 1] > landscape[current]:
            improving.append(current + 1)
        if not improving:
            break
        current = random.choice(improving)
        path.append(current)
    return current, path

```

```

# -----
# Helper: find all local maxima
# -----
def find_local_maxima(landscape):
    """
    Returns list of 0-based indices whose f-value strictly exceeds both neighbours.
    Edge positions are considered to have one neighbour only.
    """
    maxima = []
    n = len(landscape)
    for i in range(n):
        left_ok = (i == 0) or landscape[i] > landscape[i - 1]
        right_ok = (i == n - 1) or landscape[i] > landscape[i + 1]
        if left_ok and right_ok:
            maxima.append(i)
    return maxima

```

```

# -----
# Q2(a): Random Restart Hill Climbing

```

```

# -----
def random_restart_hc(landscape, num_restarts, variant='first_choice'):
    """
    Runs hill climbing from num_restarts random starting positions.

    Parameters
    -----
    landscape : list of int/float utility values
    num_restarts : int
    variant : 'first_choice' | 'stochastic'

    Returns
    -----
    best_state : 0-based index of the best terminal state found
    best_value : f(best_state)
    all_results : list of (start, terminal, path) tuples (all 0-based indices)
    """
    hc = first_choice_hc if variant == 'first_choice' else stochastic_hc

    best_state = None
    best_value = float('-inf')
    all_results = []

    for _ in range(num_restarts):
        start = random.randint(0, len(landscape) - 1)
        terminal, path = hc(landscape, start)
        val = landscape[terminal]
        all_results.append((start, terminal, path))
        if val > best_value:
            best_value = val
            best_state = terminal

    return best_state, best_value, all_results

# -----
# Q2(b): Empirical probability experiment
# -----
def empirical_probability(landscape, restart_counts, trials=100,
                        variant='first_choice', global_max_idx=GLOBAL_MAX_IDX):
    """
    For each n in restart_counts, runs 'trials' independent RRHC runs and
    returns the fraction that find the global maximum.
    """

```

```

results = {}
for n in restart_counts:
    successes = 0
    for _ in range(trials):
        _, _, all_res = random_restart_hc(landscape, n, variant)
        if any(terminal == global_max_idx for _, terminal, _ in all_res):
            successes += 1
    results[n] = successes / trials
return results

```

```

def theoretical_probability(p, restart_counts):
    """P(find global max in n restarts) = 1 - (1-p)^n"""
    return {n: 1 - (1 - p) ** n for n in restart_counts}

```

```

def derive_p_first_choice(landscape, global_max_idx=GLOBAL_MAX_IDX):
    """
    Run first-choice HC from every starting state and count how many
    lead to the global maximum. Returns p = count / len(landscape).
    """
    n = len(landscape)
    count = 0
    print("\n First-Choice HC basin analysis (all starting states):")
    print(f" {'Start (state)':>14} {'Terminal (state)':>16} {'Reaches Global Max?':>20}")
    print(" " + "-" * 54)
    for i in range(n):
        terminal, _ = first_choice_hc(landscape, i)
        reaches = terminal == global_max_idx
        if reaches:
            count += 1
        print(f" {i+1:>14} {terminal+1:>16} {'Yes' if reaches else 'No':>20}")
    p = count / n
    print(f"\n States reaching global max: {count}/{n} => p = {p:.4f}")
    return p

```

```

# -----
# Q2(c): Plateau investigation
# -----

```

```

def rrhc_with_plateau_counter(landscape, num_restarts, variant='first_choice',
                              plateau_states=None, global_max_idx=GLOBAL_MAX_IDX):
    """
    Extended RRHC that additionally counts restarts terminating on a plateau state.

```

Returns

-----

best\_state, best\_value, all\_results, plateau\_count, global\_count  
"""

if plateau\_states is None:  
 plateau\_states = []

hc = first\_choice\_hc if variant == 'first\_choice' else stochastic\_hc

best\_state = None  
best\_value = float('-inf')  
all\_results = []  
plateau\_count = 0  
global\_count = 0

for \_ in range(num\_restarts):  
 start = random.randint(0, len(landscape) - 1)  
 terminal, path = hc(landscape, start)  
 val = landscape[terminal]  
 all\_results.append((start, terminal, path))

if terminal in plateau\_states:  
 plateau\_count += 1  
if terminal == global\_max\_idx:  
 global\_count += 1

if val > best\_value:  
 best\_value = val  
 best\_state = terminal

return best\_state, best\_value, all\_results, plateau\_count, global\_count

# \_\_\_\_\_

# MAIN

# \_\_\_\_\_

def main():  
 random.seed(42) # reproducibility

# — Print landscape info \_\_\_\_\_  
print("=" \* 65)  
print(" Q2 — RANDOM RESTART HILL CLIMBING")  
print("=" \* 65)

```

print("\nLandscape:")
print(f" {'State':>6} {'f(s)':>6}")
print(" " + "-" * 16)
for i, v in enumerate(LANDSCAPE):
    print(f" {i+1:>6} {v:>6}")

# — Local maxima —————
lm = find_local_maxima(LANDSCAPE)
print(f"\nLocal maxima (0-based indices): {lm}")
print(f"Local maxima (states, 1-based): {[i+1 for i in lm]}")
print(f" Values: {[LANDSCAPE[i] for i in lm]}")
print(f" Global maximum: State {GLOBAL_MAX_IDX+1}, f = {LANDSCAPE[GLOBAL_MAX_IDX]}")

# —————
# PART (a): Run RRHC with 20 restarts under both variants
# —————
print("\n" + "=" * 65)
print(" PART (a): RRHC with 20 restarts")
print("=" * 65)

for variant in ('first_choice', 'stochastic'):
    random.seed(42)
    best_state, best_value, all_results = random_restart_hc(
        LANDSCAPE, num_restarts=20, variant=variant)

    print(f"\n Variant: {variant.upper().replace('_', '-')}")
    print(f" {'Restart':>8} {'Start':>6} {'Terminal':>9} {'f-value':>8} {'Global Max?':>12}")
    print(" " + "-" * 50)
    for r_num, (start, terminal, _) in enumerate(all_results, 1):
        gm = "Yes" if terminal == GLOBAL_MAX_IDX else "No"
        print(f" {r_num:>8} {start+1:>6} {terminal+1:>9} "
              f"{LANDSCAPE[terminal]:>8} {gm:>12}")
    print(f"\n => Best found: State {best_state+1}, f = {best_value}")

# —————
# PART (b): Empirical vs theoretical probability
# —————
print("\n" + "=" * 65)
print(" PART (b): Empirical vs Theoretical probability")
print("=" * 65)

# Derive p from first-choice HC
p = derive_p_first_choice(LANDSCAPE)

```

```

restart_counts = [1, 3, 5, 10, 20]

# Empirical
random.seed(0)
emp = empirical_probability(LANDSCAPE, restart_counts, trials=100,
                           variant='first_choice')

# Theoretical
theo = theoretical_probability(p, restart_counts)

print("\n Theoretical calculations ( $P = 1 - (1-p)^n$ ):")
for n in restart_counts:
    print(f"    n={n}>2:  $P = 1 - (1 - \{p:.4f\})^{\{n\}} = 1 - \{(1-p)^{\{n\}}\} = \{theo[n]:.6f\}$ ")

print(f"\n {'n':>4} {'Empirical P':>12} {'Theoretical P':>14} {'Difference':>11}")
print(" " + "-" * 46)
for n in restart_counts:
    diff = emp[n] - theo[n]
    print(f" {'n':>4} {emp[n]:>12.4f} {theo[n]:>14.4f} {diff:>+11.4f}")

# -----
# PART (c): Plateau investigation
# -----
print("=" * 65)
print(" PART (c): Plateau investigation (states 7 & 8 both f=17)")
print("=" * 65)

LANDSCAPE_PLATEAU = LANDSCAPE.copy()
LANDSCAPE_PLATEAU[7] = 17 # state 8 raised to 17 (was 14)
# state 7 (index 6) already = 17

PLATEAU_INDICES = [6, 7] # 0-based indices for states 7 & 8

print("\nModified landscape:")
print(f" {'State':>6} {'Original f(s)':>14} {'Modified f(s)':>14} {'Changed?':>9}")
print(" " + "-" * 48)
for i, (orig, mod) in enumerate(zip(LANDSCAPE, LANDSCAPE_PLATEAU)):
    changed = "*" if orig != mod else ""
    print(f" {i+1:>6} {orig:>14} {mod:>14} {changed:>9}")

NUM_RESTARTS = 20
TRIALS = 100

# — original landscape —————

```

```

random.seed(7)
orig_global_rates = []
orig_plateau_rates = []
for _ in range(TRIALS):
    _, _, all_res, p_cnt, g_cnt = rrhc_with_plateau_counter(
        LANDSCAPE, NUM_RESTARTS, variant='first_choice',
        plateau_states=PLATEAU_INDICES, global_max_idx=GLOBAL_MAX_IDX)
    orig_global_rates.append(g_cnt / NUM_RESTARTS)
    orig_plateau_rates.append(p_cnt / NUM_RESTARTS)
avg_orig_global = sum(orig_global_rates) / TRIALS
avg_orig_plateau = sum(orig_plateau_rates) / TRIALS

# — modified (plateau) landscape —————
random.seed(7)
mod_global_rates = []
mod_plateau_rates = []
for _ in range(TRIALS):
    _, _, all_res, p_cnt, g_cnt = rrhc_with_plateau_counter(
        LANDSCAPE_PLATEAU, NUM_RESTARTS, variant='first_choice',
        plateau_states=PLATEAU_INDICES, global_max_idx=GLOBAL_MAX_IDX)
    mod_global_rates.append(g_cnt / NUM_RESTARTS)
    mod_plateau_rates.append(p_cnt / NUM_RESTARTS)
avg_mod_global = sum(mod_global_rates) / TRIALS
avg_mod_plateau = sum(mod_plateau_rates) / TRIALS

# — single representative run (for per-restart table) —————
random.seed(42)
_, _, all_res_orig, pc_orig, gc_orig = rrhc_with_plateau_counter(
    LANDSCAPE, NUM_RESTARTS, 'first_choice', PLATEAU_INDICES, GLOBAL_MAX_IDX)
_, _, all_res_mod, pc_mod, gc_mod = rrhc_with_plateau_counter(
    LANDSCAPE_PLATEAU, NUM_RESTARTS, 'first_choice', PLATEAU_INDICES, GLOBAL_MAX_IDX)

# Per-restart detail tables
for label, results, p_cnt, g_cnt, land in [
    ("ORIGINAL landscape", all_res_orig, pc_orig, gc_orig, LANDSCAPE),
    ("MODIFIED landscape (states 7&8 both f=17)", all_res_mod, pc_mod, gc_mod,
LANDSCAPE_PLATEAU),
]:
    print(f"\n --- {label} ---")
    print(f" {'Restart':>8} {'Start':>6} {'Terminal':>9} {'f-value':>8} "
          f"{'Global Max?':>12} {'On Plateau?':>12}")
    print(" " + "-" * 66)
    for r_num, (start, terminal, _) in enumerate(results, 1):
        gm = "Yes" if terminal == GLOBAL_MAX_IDX else "No"

```

```

plt = "Yes" if terminal in PLATEAU_INDICES else "No"
print(f" {r_num:>8} {start+1:>6} {terminal+1:>9} "
      f"{land[terminal]:>8} {gm:>12} {plt:>12}")
print(f"\n Plateau terminations : {p_cnt}/{NUM_RESTARTS}")
print(f" Global max finds   : {g_cnt}/{NUM_RESTARTS}")

# Summary comparison table
print("\n --- Summary over 100 independent RRHC(20) runs ---")
print(f" {'Landscape':>30} {'Avg Global-Max Rate':>20} {'Avg Plateau Rate':>17}")
print(" " + "-" * 72)
print(f" {'Original':>30} {avg_orig_global:>20.4f} {avg_orig_plateau:>17.4f}")
print(f" {'Modified (plateau at 7&8)':>30} {avg_mod_global:>20.4f} {avg_mod_plateau:>17.4f}")

if __name__ == "__main__":
    main()

```

```

=====
Q2  RANDOM RESTART HILL CLIMBING
=====

Landscape:
  State    f(s)
  -----
    1         5
    2         8
    3         6
    4        12
    5         9
    6         7
    7        17
    8        14
    9        10
   10         6
   11        19
   12        15
   13        11
   14         8

Local maxima (0-based indices): [1, 3, 6, 10]
Local maxima (states, 1-based): [2, 4, 7, 11]
  Values: [8, 12, 17, 19]
Global maximum: State 11, f = 19

=====
PART 4 - RRHC with 20 restarts

```

```
=====
PART (a): RRHC with 20 restarts
=====
```

```
Variant: FIRST-CHOICE
```

| Restart | Start | Terminal | f-value | Global Max? |
|---------|-------|----------|---------|-------------|
| 1       | 11    | 11       | 19      | Yes         |
| 2       | 2     | 2        | 8       | No          |
| 3       | 1     | 2        | 8       | No          |
| 4       | 12    | 11       | 19      | Yes         |
| 5       | 5     | 4        | 12      | No          |
| 6       | 4     | 4        | 12      | No          |
| 7       | 4     | 4        | 12      | No          |
| 8       | 3     | 2        | 8       | No          |
| 9       | 12    | 11       | 19      | Yes         |
| 10      | 2     | 2        | 8       | No          |
| 11      | 11    | 11       | 19      | Yes         |
| 12      | 12    | 11       | 19      | Yes         |
| 13      | 9     | 7        | 17      | No          |
| 14      | 2     | 2        | 8       | No          |
| 15      | 10    | 7        | 17      | No          |
| 16      | 7     | 7        | 17      | No          |
| 17      | 1     | 2        | 8       | No          |
| 18      | 1     | 2        | 8       | No          |
| 19      | 2     | 2        | 8       | No          |
| 20      | 4     | 4        | 12      | No          |

-----  
=> Best found: State 11, f = 19

Variant: STOCHASTIC

Restart    Start    Terminal    f-value    Global Max?

|    |    |    |    |    |     |
|----|----|----|----|----|-----|
|    | 1  | 11 | 11 | 19 | Yes |
|    | 2  | 2  | 2  | 8  | No  |
|    | 3  | 1  | 2  | 8  | No  |
|    | 4  | 4  | 4  | 12 | No  |
|    | 5  | 4  | 4  | 12 | No  |
|    | 6  | 3  | 2  | 8  | No  |
|    | 7  | 11 | 11 | 19 | Yes |
|    | 8  | 12 | 11 | 19 | Yes |
|    | 9  | 10 | 11 | 19 | Yes |
| 10 | 1  | 2  | 2  | 8  | No  |
| 11 | 2  | 2  | 2  | 8  | No  |
| 12 | 4  | 4  | 4  | 12 | No  |
| 13 | 4  | 4  | 4  | 12 | No  |
| 14 | 9  | 7  | 7  | 17 | No  |
| 15 | 12 | 11 | 11 | 19 | Yes |
| 16 | 4  | 4  | 4  | 12 | No  |
| 17 | 8  | 7  | 7  | 17 | No  |
| 18 | 13 | 11 | 11 | 19 | Yes |
| 19 | 12 | 11 | 11 | 19 | Yes |
| 20 | 6  | 7  | 7  | 17 | No  |

=> Best found: State 11, f = 19

=====  
PART (b): Empirical vs Theoretical probability  
=====

First-Choice HC basin analysis (all starting states):

Start (state) Terminal (state) Reaches Global Max?

|  |  |    |    |     |
|--|--|----|----|-----|
|  |  | 1  | 2  | No  |
|  |  | 2  | 2  | No  |
|  |  | 3  | 2  | No  |
|  |  | 4  | 4  | No  |
|  |  | 5  | 4  | No  |
|  |  | 6  | 4  | No  |
|  |  | 7  | 7  | No  |
|  |  | 8  | 7  | No  |
|  |  | 9  | 7  | No  |
|  |  | 10 | 7  | No  |
|  |  | 11 | 11 | Yes |
|  |  | 12 | 11 | Yes |
|  |  | 13 | 11 | Yes |
|  |  | 14 | 11 | Yes |

States reaching global max: 4/14 => p = 0.2857

Theoretical calculations ( $P = 1 - (1-p)^n$ ):

n= 1:  $P = 1 - (1 - 0.2857)^1 = 1 - 0.714286 = 0.285714$

States reaching global max: 4/14 => p = 0.2857

Theoretical calculations ( $P = 1 - (1-p)^n$ ):

n= 1:  $P = 1 - (1 - 0.2857)^1 = 1 - 0.714286 = 0.285714$

n= 3:  $P = 1 - (1 - 0.2857)^3 = 1 - 0.364431 = 0.635569$

n= 5:  $P = 1 - (1 - 0.2857)^5 = 1 - 0.185934 = 0.814066$

n=10:  $P = 1 - (1 - 0.2857)^{10} = 1 - 0.034572 = 0.965428$

n=20:  $P = 1 - (1 - 0.2857)^{20} = 1 - 0.001195 = 0.998805$

| n  | Empirical P | Theoretical P | Difference |
|----|-------------|---------------|------------|
| 1  | 0.3000      | 0.2857        | +0.0143    |
| 3  | 0.7300      | 0.6356        | +0.0944    |
| 5  | 0.8300      | 0.8141        | +0.0159    |
| 10 | 0.9900      | 0.9654        | +0.0246    |
| 20 | 1.0000      | 0.9988        | +0.0012    |

## Part (b) – Analysis (Empirical vs. Theoretical probabilities)

### Analysis:

The empirical probabilities closely match the theoretical predictions for small restart counts ( $n = 1, 3, 5$ ), indicating that the Bernoulli model  $P = 1 - (1-p)^n$  is a good approximation. For larger  $n$  (10, 20), the empirical probabilities slightly exceed the theoretical values, showing that the finite landscape and repeated starts sometimes increase the chance of hitting the global maximum more than the idealized independent-trial assumption predicts. The remaining differences are mainly due to the limited number of trials (100) and the discrete nature of starting states, which introduces quantization effects. Overall, the agreement validates the formula while highlighting small deviations caused by practical constraints.

```
=====
PART (c): Plateau investigation (states 7 & 8 both f=17)
=====
```

Modified landscape:

| State | Original f(s) | Modified f(s) | Changed? |
|-------|---------------|---------------|----------|
| 1     | 5             | 5             |          |
| 2     | 8             | 8             |          |
| 3     | 6             | 6             |          |
| 4     | 12            | 12            |          |
| 5     | 9             | 9             |          |
| 6     | 7             | 7             |          |
| 7     | 17            | 17            |          |
| 8     | 14            | 17            | *        |
| 9     | 10            | 10            |          |
| 10    | 6             | 6             |          |
| 11    | 19            | 19            |          |
| 12    | 15            | 15            |          |
| 13    | 11            | 11            |          |
| 14    | 8             | 8             |          |

--- ORIGINAL landscape ---

| Restart | Start | Terminal | f-value | Global Max? | On Plateau? |
|---------|-------|----------|---------|-------------|-------------|
| -----   |       |          |         |             |             |
| 1       | 11    | 11       | 19      | Yes         | No          |
| 2       | 2     | 2        | 8       | No          | No          |
| 3       | 1     | 2        | 8       | No          | No          |
| 4       | 12    | 11       | 19      | Yes         | No          |
| 5       | 5     | 4        | 12      | No          | No          |
| 6       | 4     | 4        | 12      | No          | No          |
| 7       | 4     | 4        | 12      | No          | No          |
| 8       | 3     | 2        | 8       | No          | No          |
| 9       | 12    | 11       | 19      | Yes         | No          |
| 10      | 2     | 2        | 8       | No          | No          |
| 11      | 11    | 11       | 19      | Yes         | No          |
| 12      | 12    | 11       | 19      | Yes         | No          |
| 13      | 9     | 7        | 17      | No          | Yes         |
| 14      | 2     | 2        | 8       | No          | No          |
| 15      | 10    | 7        | 17      | No          | Yes         |
| 16      | 7     | 7        | 17      | No          | Yes         |
| 17      | 1     | 2        | 8       | No          | No          |
| 18      | 1     | 2        | 8       | No          | No          |
| 19      | 2     | 2        | 8       | No          | No          |
| 20      | 4     | 4        | 12      | No          | No          |

Plateau terminations : 3/20

Global max finds : 5/20

```
Plateau terminations : 3/20
Global max finds      : 5/20
```

```
--- MODIFIED landscape (states 7&8 both f=17) ---
```

| Restart | Start | Terminal | f-value | Global Max? | On Plateau? |
|---------|-------|----------|---------|-------------|-------------|
| 1       | 4     | 4        | 12      | No          | No          |
| 2       | 9     | 8        | 17      | No          | Yes         |
| 3       | 10    | 8        | 17      | No          | Yes         |
| 4       | 1     | 2        | 8       | No          | No          |
| 5       | 9     | 8        | 17      | No          | Yes         |
| 6       | 4     | 4        | 12      | No          | No          |
| 7       | 12    | 11       | 19      | Yes         | No          |
| 8       | 11    | 11       | 19      | Yes         | No          |
| 9       | 12    | 11       | 19      | Yes         | No          |
| 10      | 9     | 8        | 17      | No          | Yes         |
| 11      | 7     | 7        | 17      | No          | Yes         |
| 12      | 4     | 4        | 12      | No          | No          |
| 13      | 8     | 8        | 17      | No          | Yes         |
| 14      | 10    | 8        | 17      | No          | Yes         |
| 15      | 5     | 4        | 12      | No          | No          |
| 16      | 13    | 11       | 19      | Yes         | No          |
| 17      | 14    | 11       | 19      | Yes         | No          |
| 18      | 1     | 2        | 8       | No          | No          |
| 19      | 13    | 11       | 19      | Yes         | No          |
| 20      | 13    | 11       | 19      | Yes         | No          |

Ln 18, Col 8 Spaces: 4 UTF-8 CRLF { } Py

```
Plateau terminations : 7/20
Global max finds      : 7/20
```

```
--- Summary over 100 independent RRHC(20) runs ---
```

|                           | Landscape | Avg Global-Max Rate | Avg Plateau Rate |
|---------------------------|-----------|---------------------|------------------|
| Original                  |           | 0.2635              | 0.2820           |
| Modified (plateau at 7&8) |           | 0.2635              | 0.2820           |

## Part (c) – Plateau Analysis

### Analysis:

Introducing a plateau by setting states 7 and 8 to the same f-value (17) increases the number of restarts that terminate on the plateau from 3/20 to 7/20, reducing the relative proportion of restarts that reach the global maximum. While more restarts can eventually compensate, the plateau remains a persistent trap, meaning some restarts will always terminate there rather than at state 11. The experimental numbers show that even with 20 restarts, only 7 out of 20 reach the global maximum after the plateau modification, demonstrating that plateaus slow convergence but do not completely prevent eventual discovery if the restart budget is increased. This confirms that RRHC mitigates, but does not eliminate, the effect of flat regions in the landscape.